Khant Thu Aung / 105292912

10.1C Iteration 8

TakeCommand.cs

```
using System.ComponentModel;

namespace SwinAdventure;

public class TakeCommand : Command
{
    public TakeCommand() : base(new string[]{"pickup","take"})
    {
    }

    public override string Execute(Player p, string[] text)
    {
        if (text.Length != 2 && text.Length != 4)
            return "I don't understand what you want to take.";

        string itemId = text[1];

        if (text.Length == 2)
        {
            // take from current location
            GameObject item = p.Location.Inventory.Take(itemId);
            if (item == null)
                return $"There is no {itemId} here.";
```

```
    if (item is Item takenItem)
    {
        p.Inventory.Put(takenItem);
        return $"You took the {takenItem.Name} from {p.Location.Name}";
    }
    return $"{itemId} is not an item you can take.";
}
else if (text.Length == 4 && text[2].ToLower() == "from")
{
    string containerId = text[3];
    GameObject container = p.Locate(containerId);

    if (container is IHaveInventory hasInventory)
    {
        GameObject item = hasInventory.Inventory.Take(itemId);
        if (item == null)
            return $"There is no {itemId} in the {container.Name}.";

        if (item is Item takenItem)
        {
            p.Inventory.Put(takenItem);
            return $"You took the {takenItem.Name} from the {container.Name}.";
        }
        return $"{itemId} is not an item you can take.";
    }
```

```
            return $"I can't find the {containerId}.";
        }
        return "I don't understand how to take like that.";
    }
}
```

PutCommand.cs

```
namespace SwinAdventure;


public class PutCommand : Command
{
    public PutCommand() : base(new string[] { "put", "drop" }) { }


    public override string Execute(Player p, string[] text)
    {
        if (text.Length == 2)
        {
            // put item / drop item => drop to ground (location)
            string itemId = text[1];
            Item item = p.Inventory.Take(itemId);
            if (item == null)
                return $"You don't have {itemId}.";


            p.Location.Inventory.Put(item);
            return $"You dropped {item.Name} at {p.Location.Name}.";
        }
```

```
        if (text.Length == 4 && text[2].ToLower() == "in")
        {
            string itemId = text[1];
            string containerId = text[3];

            // Locate container (bag, location, etc.)
            IHaveInventory container = FetchContainer(p, containerId);
            if (container == null)
                return $"I can't find the {containerId}.";

            Item item = p.Inventory.Take(itemId);
            if (item == null)
                return $"You don't have {itemId}.";

            container.Inventory.Put(item);
            return $"You put the {item.Name} in the {containerId}.";
        }

        return "I don't know how to put like that.";
    }
    public IHaveInventory FetchContainer(Player p, string id)
    {
        GameObject obj = p.Locate(id);
        return obj as IHaveInventory;
    }
}
```

CommandProcessor.cs

```
namespace SwinAdventure;


public class CommandProcessor
{
   private List<Command> _commands;


   public CommandProcessor()
   {
      _commands = new List<Command>
      {
         new LookCommand(),
         new MoveCommand(),
         new TakeCommand(),
         new PutCommand()
      };
   }


   public string ExecuteCommand(Player player, string[] input)
   {
      if (input.Length == 0 || string.IsNullOrWhiteSpace(input[0]))
         return "No command given.";


      string commandWord = input[0].ToLower();


      foreach (Command cmd in _commands)
```

```
        {
            if (cmd.AreYou(commandWord))
                return cmd.Execute(player, input);
        }
        return $"I don't understand '{commandWord}'.";
    }
}
```

Program.cs

```
namespace SwinAdventure;


public class Program
{
    public static void Main()
    {
        string playerName;
        string playerDescription;
        CommandProcessor processor = new CommandProcessor();


        Console.WriteLine("Enter your name, Adventurer!");
        Console.Write("Name -> ");
        playerName = Console.ReadLine();
        Console.WriteLine("Now how do you describe yourself, Adventurer!");
        Console.Write("Description -> ");
        playerDescription = Console.ReadLine();
```

```
Location spawnPoint = new(new string[]{"gate"},"Front Gate", "a Front Gate of the theme park.");

Player me = new(playerName, playerDescription, spawnPoint);

me.Location = spawnPoint;


//locations

Location ferrisWheel = new(new string[]{"ferris"},"Ferris Wheel", "a ferris wheel area that many people are lining to ride.");

Location jungleRide = new(new string[]{"jungle"},"Jungle Ride", "a water ride with the vibe full of jungle.\n You can even hear the bird noise from the front.");

Location hauntedHouse = new(new string[]{"haunted"},"Haunted House", "a really scary haunted house that many screaming voices are coming from there.");

Location iceCreamShop = new(new string[]{"shop"},"Ice-cream Shop", "an icecream shop that many parents are lining up for their chilren.");

Location boatRace = new(new string[]{"boat"},"Boat Race", "a boat race where you can participate.\nA race just began.");


//exits for each location

//from spawn

Path spawnEast = new(new string[] { "east" ,"e"}, "Concerete Path", "a path leadig to ferris wheel area", ferrisWheel);

spawnPoint.AddPath(spawnEast);


Path ferrisEast = new(new string[] { "east" ,"e"}, "Concerete Path", "a path leading to Jungle Ride area", jungleRide);

Path ferrisWest = new(new string[] { "west","w" }, "Concerete Path", "a path leading back to gate. ", spawnPoint);

ferrisWheel.AddPath(ferrisEast);

ferrisWheel.AddPath(ferrisWest);
```

```
Path jungleWest = new(new string[] { "west" ,"w"}, "Concerete Path", "a path leading back to Ferris Wheel area", ferrisWheel);

Path jungleNorth = new(new string[] { "north" ,"n"}, "Dirt Path", "a path leading to Haunted House area", hauntedHouse);

jungleRide.AddPath(jungleWest);

jungleRide.AddPath(jungleNorth);

Path hauntEast = new(new string[] { "east" ,"e"}, "Small Bridge", "a path leading to Ice Cream Shop", iceCreamShop);

Path hauntSouth = new(new string[] { "south" ,"s" }, "Dirt Path", "a path leading back to Jungle Ride Area", jungleRide);

Path hauntSe = new(new string[] { "southeast" ,"se" }, "Stone Path", "a path leading to Boat Racing Area", boatRace);

hauntedHouse.AddPath(hauntEast);

hauntedHouse.AddPath(hauntSouth);

hauntedHouse.AddPath(hauntSe);

Path iceSouth = new(new string[] { "south","s" }, "Concerete Path", "a path leading to Boat Racing Area", boatRace);

Path iceWest = new(new string[] { "west","w" }, "Small Bridge", "a path leading back to Haunted House Area", hauntedHouse);

iceCreamShop.AddPath(iceSouth);

iceCreamShop.AddPath(iceWest);

Path boatNw = new(new string[] { "northwest","nw" }, "Stone Path", "a path leading to Haunted House Area", hauntedHouse);

Path boatNorth = new(new string[] { "north" ,"n"}, "Concerete Path", "a path leading to Ice Cream Area", iceCreamShop);

boatRace.AddPath(boatNw);

boatRace.AddPath(boatNorth);


//items
```

```
Item phone = new(new string[] { "phone", "modern phone" }, "A modern phone", "A phone that you have been using for 2 years.");

Item wallet = new(new string[] { "wallet", "old wallet" }, "An old wallet", "A wallet that your dad gifted on your 10th birthday.");

Bag bag = new(new string[] { "bag", "school bag" }, "A school bag", "A blue school bag that you take everywhere you go.");

Item ring = new(new string[] { "ring", "gold ring" }, "A gold ring", "A high value ring that's been passed down to you");


Bag oldBag = new(new string[] { "oldbag", "torn bag" }, "a torn bag", "A torn bag someone left behind");

Item ticket = new(new string[] { "ticket", "entry ticket" }, "An entry ticket", "A ticket to the theme park.");

Item miniFan = new(new string[] { "fan", "small fan" }, "A small fan", "A small fan that can make you cool");

Item coin = new(new string[] { "coin", "bronze coin" }, "A bronze coin", "A bronze coin that someone dropped");

Item flower = new(new string[] { "flower", "yellow flower" }, "A yellow flower", "A yellow flower growing near the Jungle Ride");

Item skull = new(new string[] { "skull", "plastic skull" }, "A plastic skull", "A plastic scary skull that you found in Haunted House");

Item iceCream = new(new string[] { "icecream", "vanilla icecream" }, "A vanilla icecream", "People say this vanilla icecream is the best");

Item miniBoat = new(new string[] { "miniboat", "red miniboat" }, "A red miniboat", "Staffs are giving out miniboat for the kids");

Item book = new(new string[] { "book", "small book" }, "a small book", "a book that is inside a torn bag");


//player initial inventory

me.Inventory.Put(phone);
```

```
me.Inventory.Put(wallet);
me.Inventory.Put(bag);
bag.Inventory.Put(ring);
oldBag.Inventory.Put(book);

//locations inventories
//spawn
spawnPoint.Inventory.Put(ticket);

//ferrisWheel
ferrisWheel.Inventory.Put(miniFan);
ferrisWheel.Inventory.Put(oldBag);

//jungle
jungleRide.Inventory.Put(coin);
jungleRide.Inventory.Put(flower);

//haunted house
hauntedHouse.Inventory.Put(skull);

//icecream
iceCreamShop.Inventory.Put(iceCream);

//boat race
boatRace.Inventory.Put(miniBoat);
```

```
        Console.WriteLine("Welcome to Swin Adventure!\nYou have arrived at the front gate of the theme park.");
        while (true)
        {
            string command = "";
            Console.Write("Command -> ");
            command = Console.ReadLine();
            string[] parts = command.ToLower().Split(' ');

            if (parts[0] == "exit" || parts[0] == "end")
            {
                Console.WriteLine("Bye Adventurer!");
                return;
            }
            Console.WriteLine(processor.ExecuteCommand(me, parts));
        }
    }
}
```

TestCommands.cs

```
using NUnit.Framework.Legacy;
using SwinAdventure;
namespace UnitTest;
[TestFixture]
public class CommandTests
{
    private Player player;
```

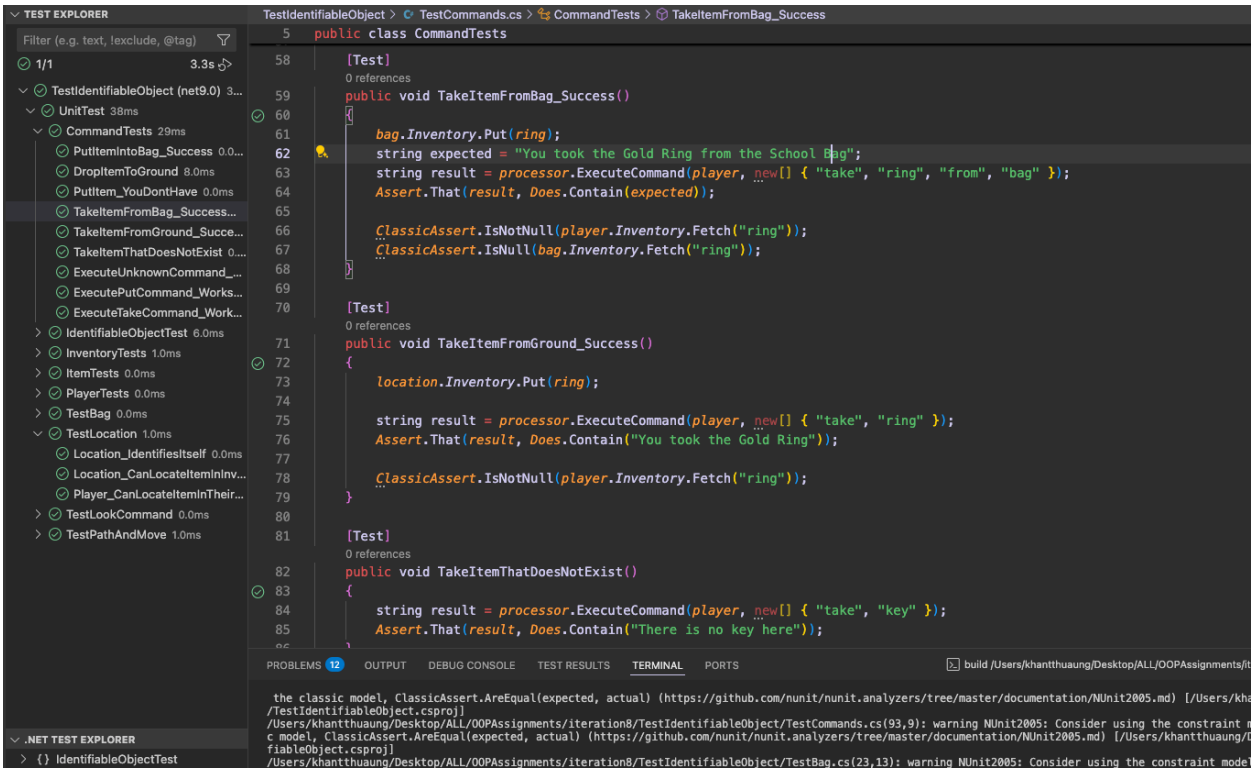
```
private Location location;
private Bag bag;
private Item phone;
private Item ring;
private CommandProcessor processor;

[SetUp]
public void Setup()
{
    location = new Location(new string[]{"ground"},"ground", "The theme park entrance.");
    player = new Player("Alex", "an adventurer", location);
    processor = new CommandProcessor();

    // Items
    phone = new Item(new[] { "phone" }, "Modern Phone", "Your personal smartphone.");
    ring = new Item(new[] { "ring" }, "Gold Ring", "A precious gold ring.");

    // Bag setup
    bag = new Bag(new[] { "bag" }, "School Bag", "A small school bag.");
    player.Inventory.Put(phone);
    player.Inventory.Put(bag);
}
    [Test]
public void PutItemIntoBag_Success()
{
    string result = processor.ExecuteCommand(player, new[] { "put", "phone", "in", "bag" });
```

```
    Assert.That(result, Does.Contain("You put the Modern Phone in the bag"));

    // Item should now be in the bag
    ClassicAssert.IsNull(player.Inventory.Fetch("phone"));
    ClassicAssert.IsNotNull(bag.Inventory.Fetch("phone"));
}

[Test]
public void DropItemToGround()
{
    string result = processor.ExecuteCommand(player, new[] { "put", "phone" });
    Assert.That(result, Does.Contain("You dropped Modern Phone"));

    ClassicAssert.IsNull(player.Inventory.Fetch("phone"));
    ClassicAssert.IsNotNull(location.Inventory.Fetch("phone"));
}

[Test]
public void PutItem_YouDontHave()
{
    string result = processor.ExecuteCommand(player, new[] { "put", "wallet" });
    Assert.That(result, Does.Contain("You don't have wallet"));
}

[Test]
public void TakeItemFromBag_Success()
```

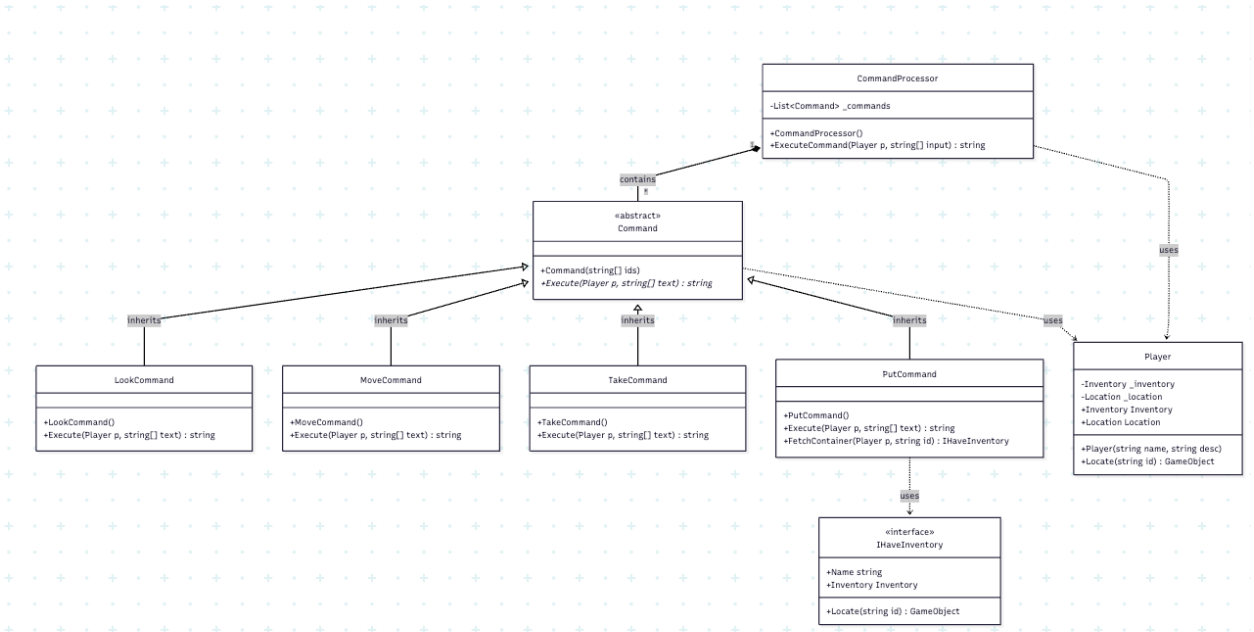
```
{
    bag.Inventory.Put(ring);
    string expected = "You took the Gold Ring from the School Bag";
    string result = processor.ExecuteCommand(player, new[] { "take", "ring", "from", "bag" });
    Assert.That(result, Does.Contain(expected));

    ClassicAssert.IsNotNull(player.Inventory.Fetch("ring"));
    ClassicAssert.IsNull(bag.Inventory.Fetch("ring"));
}

[Test]
public void TakeItemFromGround_Success()
{
    location.Inventory.Put(ring);

    string result = processor.ExecuteCommand(player, new[] { "take", "ring" });
    Assert.That(result, Does.Contain("You took the Gold Ring"));

    ClassicAssert.IsNotNull(player.Inventory.Fetch("ring"));
}

[Test]
public void TakeItemThatDoesNotExist()
{
    string result = processor.ExecuteCommand(player, new[] { "take", "key" });
    Assert.That(result, Does.Contain("There is no key here"));
```

```
}



[Test]
public void ExecuteUnknownCommand_ReturnsError()
{
   string result = processor.ExecuteCommand(player, new[] { "fly", "north" });
   ClassicAssert.AreEqual("I don't understand 'fly'.", result);
}

[Test]
public void ExecutePutCommand_Works()
{
   string result = processor.ExecuteCommand(player, new[] { "put", "phone" });
   Assert.That(result, Does.Contain("You dropped"));
}

[Test]
public void ExecuteTakeCommand_Works()
{
   location.Inventory.Put(phone);
   string result = processor.ExecuteCommand(player, new[] { "take", "phone" });

   Assert.That(result, Does.Contain("You took the Modern Phone"));
   ClassicAssert.IsNotNull(player.Inventory.Fetch("phone"));
}
```

}


TEST EXPLORER
TestIdentifiableObject > TestCommands.cs > CommandTests > TakeItemFromBag_Success
public class CommandTests
[Test]
public void TakeItemFromBag_Success()
bag.Inventory.Put(ring);
string expected = "You took the Gold Ring from the School Bag";
string result = processor.ExecuteCommand(player, new[] { "take", "ring", "from", "bag" });
Assert.That(result, Does.Contain(expected));
ClassicAssert.IsNotNull(player.Inventory.Fetch("ring"));
ClassicAssert.IsNull(bag.Inventory.Fetch("ring"));
[Test]
public void TakeItemFromGround_Success()
location.Inventory.Put(ring);
string result = processor.ExecuteCommand(player, new[] { "take", "ring" });
Assert.That(result, Does.Contain("You took the Gold Ring"));
ClassicAssert.IsNotNull(player.Inventory.Fetch("ring"));
[Test]
public void TakeItemThatDoesNotExist()
string result = processor.ExecuteCommand(player, new[] { "take", "key" });
Assert.That(result, Does.Contain("There is no key here"));
PROBLEMS 12
OUTPUT
DEBUG CONSOLE
TEST RESULTS
TERMINAL
PORTS
.NET TEST EXPLORER
IdentifiableObjectTest

UML Diagram


CommandProcessor
-List<Command> _commands
+CommandProcessor()
+ExecuteCommand(Player p, string[] input) : string
contains
«abstract»
Command
+Command(string[] ids)
+Execute(Player p, string[] text) : string
uses
inherits
LookCommand
+LookCommand()
+Execute(Player p, string[] text) : string
MoveCommand
+MoveCommand()
+Execute(Player p, string[] text) : string
TakeCommand
+TakeCommand()
+Execute(Player p, string[] text) : string
PutCommand
+PutCommand()
+Execute(Player p, string[] text) : string
+FetchContainer(Player p, string id) : IHaveInventory
Player
-Inventory _inventory
-Location _location
+Inventory Inventory
+Location Location
+Player(string name, string desc)
+Locate(string id) : GameObject
«interface»
IHaveInventory
+Name string
+Inventory Inventory
+Locate(string id) : GameObject

Sequence Diagram